

EEE4114F: ML Tutorial 1

Jarryd Son

Question 1

Suppose you have been asked to develop a system that takes images of a tube containing two-phase flows (e.g. a mixture of a vapour and liquid. See Figure 1 for an illustration) in order to determine the type of flow (bubbly, slug etc.) as well as predict a quantity known as vapour quality (how much of a mixture is a vapour).

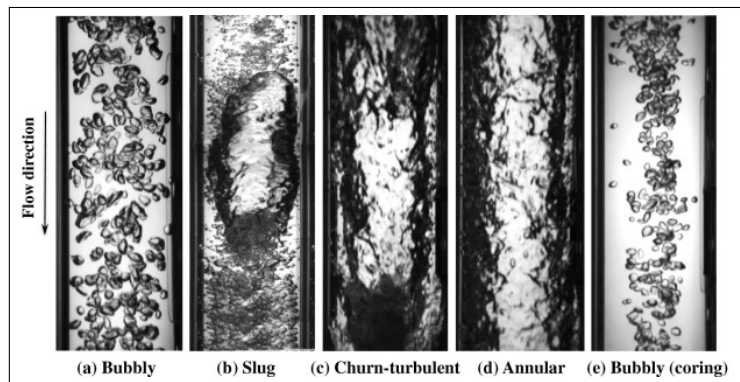


Figure 1: An example of images that indicated different types of two-phase flow.

The input data consists of greyscale images that have a resolution of $320\text{ px} \times 640\text{ px}$. The target data includes text for each class as well as the value of the vapour quality measured during the capturing of each image e.g. $y = (\text{"bubbly"}, 0.7)$. Answer the following questions:

- a. Is this a supervised learning/unsupervised learning/reinforcement learning scenario?

Solution: It is supervised learning, as you have access to ground truth data related to the input data.

- b. How many input features would be used by your models?

Solution: The input data are images. Each pixel of a given image is a feature therefore there are $320 \times 640 = 204,800$ number of features

- c. What types of learning task need to be solved? What aspects of the problem relate to each task? What component of the target data would you use for each task?

Solution:

Classification Identifying the different flows is a classification task. The text label of the flow name would be used for this.

Regression Estimating the vapour quality is a regression task. The numerical vapour quality value would be used for this.

- d. The differences between the different flows are not well defined and scientists have asked you to find different ways of grouping the flows based only on the images. What kind of learning scenario is this?

Solution: Unsupervised learning, as there is no ground truth data available.

- e. You train a model that produces the training and validation loss curves as shown in Figure 2. For each of the marked points (A/B/C/D) during the training process describe how you

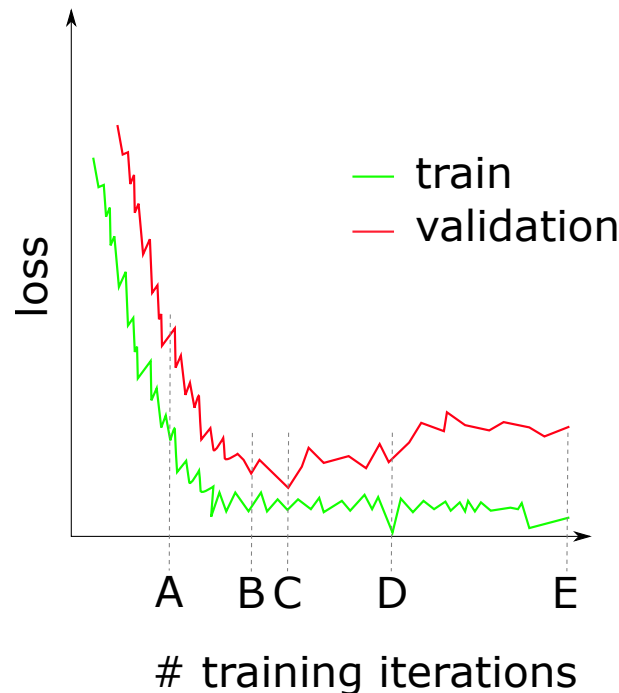


Figure 2: Loss curves during the training of a model

would expect the model to perform, and indicate which would be the point at which the model parameters would likely result in the best performance in the real world.

Solution:

- A** At this stage the model is underfit, as both the seen and unseen data have high loss values, which would indicate poor performance in the real world.
- B** At this stage both the seen and unseen data are low which indicates the model is well fit and it would perform well in the real world.
- C** At this stage both the seen and unseen data are low which indicates the model is well fit and it would perform well in the real world. In this case it indicates the lowest validation loss and is likely going to be the best performing parameters.
- D** At this stage the validation loss has started to increase even though the training loss is at it's lowest value. This is an indication that the model is overfitting to the training data and would not generalize well to unseen data even though the training loss is at it's lowest point.
- E** Similar to D. The model is overfit, it just happens to be the end of training, and it is an easy mistake to simply take the parameters at the end of training when implementing the model which would result in poor performance.

Question 2

Suppose you have a new low-cost, analogue pressure sensor, but there is no datasheet available and you do not know how the sensor's output voltage relates to pressure. You decide to try and characterise it by measuring the output voltage from the low-cost sensor with a multimeter along with pressure readings from a known sensor.

You collect the following data where each tuple represents an input-output pair (x_i, y_i) .

$$\mathcal{D} = \{(15, 5.2), (20, 6.3), (25, 7.8)\}$$

You then decide to fit a linear regression model of the form $h_{\theta}(x) = \theta_0 + \theta_1 x$ to it using gradient descent optimization. The model's parameters are randomly initialised such that $\theta_0 = 0.4, \theta_1 = 0.2$. The objective function, J , you choose to optimize is given by the mean square error:

$$J = \frac{1}{2N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i)^2$$

Where N is the number of examples in a batch. Answer the following questions:

- a. For this scenario what is the input feature and what is the output variable?

Solution: The input feature is pressure recorded by the known sensor and the output variable is the output voltage measured across the low-cost sensor.

- b. Calculate the initial loss, J , for the full batch of data.

Solution:

First you might want to calculate the output response for each input:

$$h_{\theta}(x_1) = 0.4 + 0.2(15) = 3.4$$

$$h_{\theta}(x_2) = 0.4 + 0.2(20) = 4.4$$

$$h_{\theta}(x_3) = 0.4 + 0.2(25) = 5.4$$

Then calculating, J :

$$\begin{aligned} J &= \frac{1}{2N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i)^2 \\ &= \frac{1}{2(3)} ((3.4 - 5.2)^2 + (4.4 - 6.3)^2 + (5.4 - 7.8)^2) \\ &= 2.10 \text{ V}^2 \end{aligned}$$

- c. Write down the expressions for the gradients of the objective function with respect to the model's parameters.

Solution:

$$\begin{aligned} \frac{\partial J}{\partial \theta_0} &= \frac{1}{3} \left(\sum_{i=1}^3 ((\theta_0 + \theta_1 x_i) - y_i) \right) \\ \frac{\partial J}{\partial \theta_1} &= \frac{1}{3} \left(\sum_{i=1}^3 ((\theta_0 + \theta_1 x_i) - y_i) x_i \right) \end{aligned}$$

- d. Calculate the gradient for θ_0

Solution:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{3}((3.4 - 5.2) + (4.4 - 6.3) + (5.4 - 7.8)) = -2.03$$

- e. Calculate the gradient for θ_1

Solution:

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{3}((3.4 - 5.2)(15) + (4.4 - 6.3)(20) + (5.4 - 7.8)(25)) = -41.66$$

- f. If the learning rate is, $\eta = 0.001$, calculate the new model parameters (round to three decimal places) after a single update.

Solution:

$$\begin{aligned}\theta_0 &:= \theta_0 - \eta \frac{\partial J}{\partial \theta_0} \\ &:= 0.4 - 0.001(-2.03) \\ &:= 0.402 \\ \theta_1 &:= \theta_1 - \eta \frac{\partial J}{\partial \theta_1} \\ &:= 0.2 - 0.001(-41.66) \\ &:= 0.242\end{aligned}$$

- g. You find that your model takes too long to train so you increase the learning rate to $\eta = 0.1$. What is the danger of increasing the learning rate too much?

Solution: If the learning rate is too high it could result in parameter updates that become unstable as they jump around the loss curve and they may never converge.

- h. You start to wonder if there are other phenomena that might impact this sensor e.g. perhaps temperature influences the sensor readings.
(i) How could you modify the model to incorporate one additional feature? Write down the new form for $h_\theta(x)$.

Solution:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- (ii) How would adding this additional feature impact the variance of the model?

Solution: The variance would increase. In this context of machine learning models, variance usually refers to how the model $h_\theta(x)$ would change if it is presented with different training data. With an increase in the number of features and model parameters, small changes in the training data would result in larger changes in the model's output.

- i. You decide you need more data and collect more of it to increase the number of training examples. You train your model again until the training loss is low enough. You use this trained model in your final system that outputs a pressure value based on the sensor reading.

- (i) Your model does not seem to output accurate values when you implement it in the real world. Why is it not performing as you expected?

Solution: Training loss is an optimistically biased estimate of the model's performance as it is calculated using already seen data and does not provide a good estimate of how the model would generalize to unseen data. There should have been some data that was not used during training to use as an unseen test set to provide an unbiased estimate of the model's performance.

- (ii) You perform a three-way holdout sampling technique multiple times, but your validation estimates are a poor estimate of the test performance. What resampling technique could you use that would improve this? Explain how it works and why it should produce better estimates.

Solution: Cross-validation (or k-fold cross-validation) would be an improvement. Three-way holdout does a random split of the data to provide one set for calculating parameter updates, one set used only for validation of the model's performance to compare with other models and finally one test set to estimate generalization performance. The problem with this is that even if the training process is repeated multiple times on different random splits there is no guarantee that some validation data will not be used multiple times. k-fold cross-validation repeats the splitting process k times and ensures that the validation data on each split is independent. In this way there would be multiple validation results, each on completely different examples which provides a better estimate than before.

- j. If it turns out that both features have a non-negligible impact on the sensor's output would it be more appropriate to use l_1 or l_2 penalties to regularize the objective function? Explain your answer.

Solution: In this case l_2 would make more sense. l_1 regularization could result in the parameters for the features being reduced to zero. Considering both features have a non-negligible impact, completely losing the information provided by these features would result in higher bias. l_2 regularization would keep both parameters, but constrain them to small values, which would decrease the variance of the model.

END